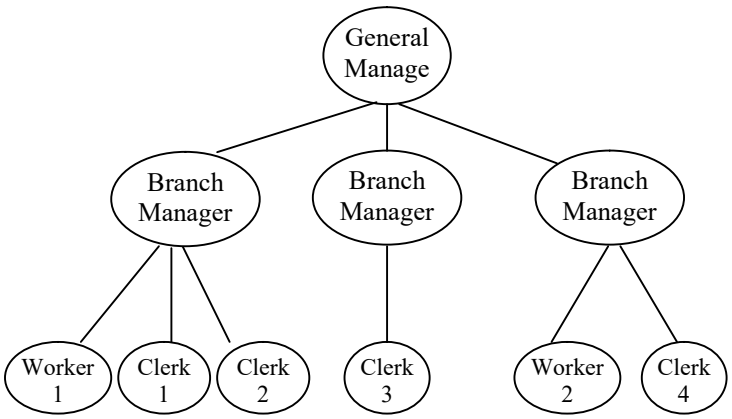
TREES

13.1 TREES

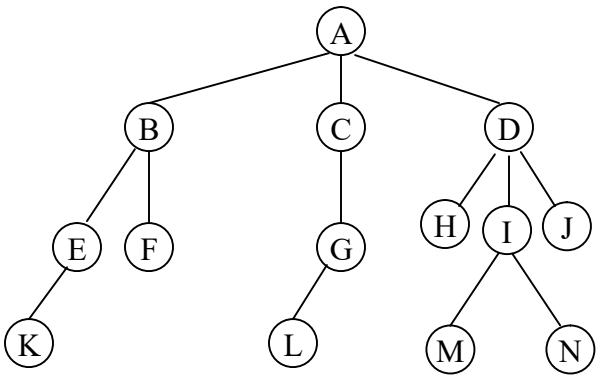
A tree is a non linear data structure where items are having hierarchical relationships. It is a data structure in which each element is attached to one or more elements directly beneath it.

The basic difference between a natural tree and a tree in data structure is that natural tree grows from ground to the air where as tree in data structure grows from top to bottom.

For Example: The Organizational tree structure of a company which shows the hierarchical relationship between the superiors and subordinates.

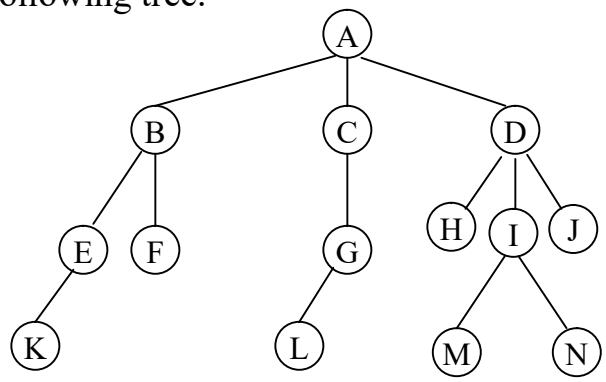


An example of general tree with nodes:



13.2 Tree Terminologies

There are number of terms associated with the trees, which can be explained with the help of a following tree.



1. **Node:** Each element of a tree is called a node. It is the basic structure in a tree. There are 14 nodes in the above tree.

2. **Root**: It is specially designated node in a tree. It is the first node in the hierarchical arrangement of data items. In the above, A is the root node.

3. **Parent Node**: Parent of a node is the immediate predecessor of a node. For example, B is the parent of E and F.

4. **Child Node**: Each immediate successor of a node is referred as child. For example, H, I, J are child nodes of D.

5. **Siblings**: The Child nodes belong to a single parent, are known as siblings. For example, E and F are siblings of B.

6. **Degree of Node**: The number of sub-tress of a given Node, is called the degree of that node. For example, degree of B is 2, degree of G is 1 and degree of F is 0.

7. **Degree of a Tree**: The maximum degree of nodes in a given tree is called degree of the tree. In the above tree, as the maximum degree of nodes A and D is 3, so the degree of tree is 3.

8. **Terminal Node:** A node with degree zero is called leaf node terminal/leaf node. In the tree above, K, F, L, H, M, N, J are terminal or leaf nodes.

9. **Level**: The entire tree structure is leveled in such a way that the root node is at level 0, its immediate child nodes are at level 1 and their immediate child nodes at level 2 and so on. In general, if a node is at level N, then its child nodes will be at level N+1. In the above tree, there are 4 levels.

10. **Edge**: It is a connecting line between two nodes. For example BE is an edge.

11. **Path**: Path is a sequence of consecutive edges from source node to destination node. In the above tree, the path between A and N is given by node pairs, (A, D), (D, I) and (I, N).

12. **Depth**: The depth of Node X is the length of unique path from root to Node X. Thus root is at depth 0. In the above tree, depth of E is 2 and depth of B is 1.

13. **Height** : The height of Node X is the longest path from the Node X to the leaf node. So all the leaf nodes are at height 0. In the above tree, height of B is 2 and height of J is 0.

14. **Ancestor and descendant:** If there is a path from node N_1 to N_2, then N_1 is an ancestor of N_2 and N_2 is descendant of N_1. For example, In the above tree A is the ancestor of G and C & C and K is the descendant of B and E.

15. **Forest**: A set of trees is called a forest.

Note:

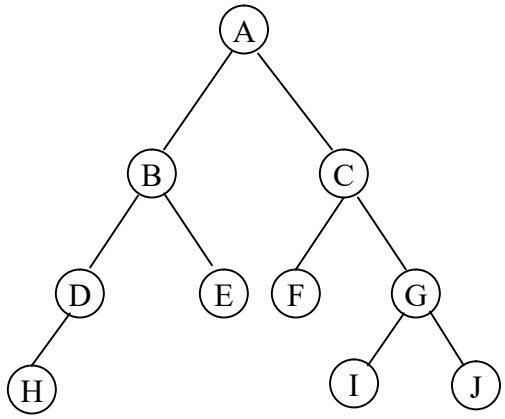
1. Root Node does not have a parent node
2. A node that has no children is called leaf node.
3. A tree having N nodes would have N-1 edges.
4. There will be exactly one path between the root and any other node in the tree.
5. The nodes with at least one child are called as non-terminal/internal nodes.
6. Terminal nodes are called external nodes/leaf nodes.

13.3 VARIATIONS IN TREE STRUCTURE

13.3.1 Binary Tree

A binary tree T is defined as a finite set of data items, called nodes, such that:

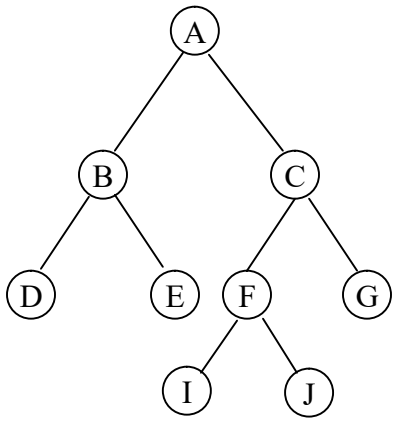
1. T is empty, called the *null tree* or *empty tree* or
2. T contains a distinguished node R, called the *root* of T, and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2 which are called *left sub-tree* and the *right sub-tree* respectively.



If T_1 is non empty, then its root is called the left successor of R. If T_2 is non empty, then its root is called the right successor of R.

13.3.2 Strictly Binary Tree:

If every non leaf node in a binary tree has non empty left and right sub-trees, the tree is termed as a strictly binary tree. A strictly binary tree with n leaves always contains 2n – 1 nodes.



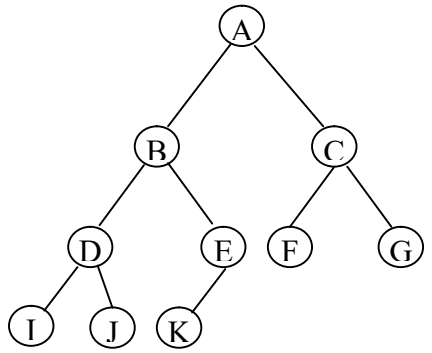
13.3.3 Complete Binary Tree:

A binary tree T is said to be *complete* if all its levels, except possibly the last level, have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible.

13.3.4 Almost Complete Binary Tree:

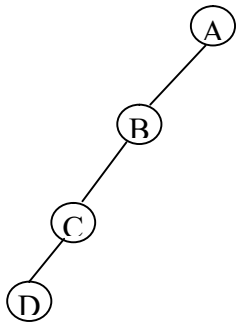
A binary tree of depth *d* is an almost complete binary tree if:

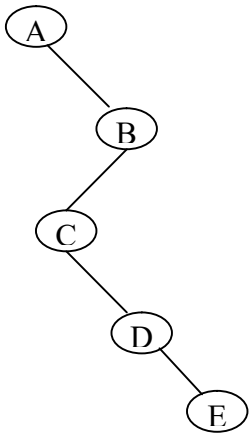
1. Each leaf in the tree is either at level *d* of at level *d*-1
2. For any node 'n' in the tree with a right descendant at level *d,* all the left descendants of 'n' that are leaves are also at level *d*.



13.3.5 Skew Binary Tree:

A tree having only one path which contains h + 1 number of nodes if 'h' is the height.





BINARY TREE

14.1 Memory Representation of Binary Tree:

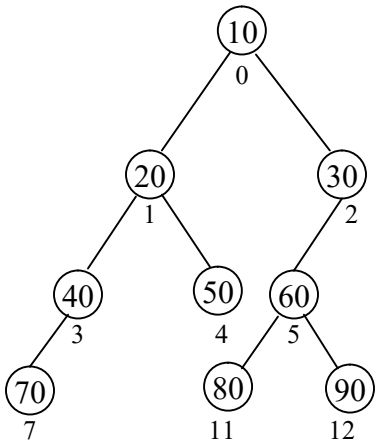
There are two common methods used for representing a binary tree.

1. Sequential Representation (using array)
2. Linked Representation(using linked list with pointers)

14.2 Array Representation of Binary Tree

A one dimensional array can be used to represent a binary tree in the memory. Here we consider that root is present at index 0. Assume tree[N] is an array representing binary tree. If '*i*' be index, then

1. Root node is stored at index 0 *i.e.* Root is at tree [0]
2. Left child is stored at tree [2 * *i* + 1]
3. Right child is stored at tree [2 * *i* + 2]



10	20	30	40	50	60		70				80	90								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

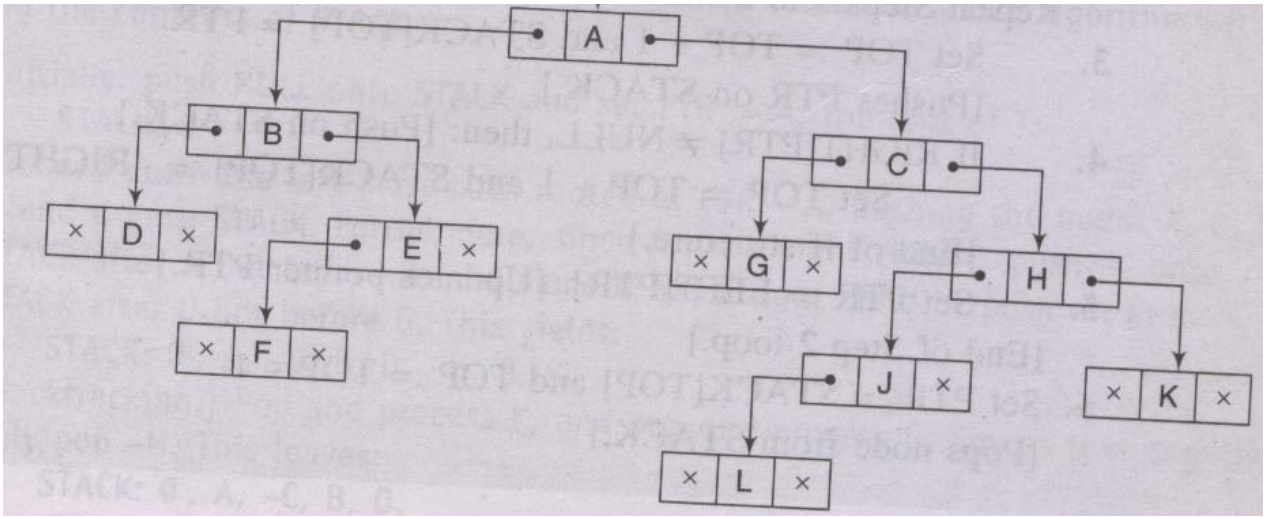
14.3 Linked Representation of Binary Tree

A binary tree can be represented by using linked list, where each node is divided into 3 parts.

1. Info : It is used to store the information
2. Left : It is a pointer that holds the address of left child
3. Right : It is a pointer that holds the address of right child

Left	Info	Right

If any sub tree of a node is empty, then its corresponding pointer of the (*i.e.* either left or right contains NULL indicating absence of the sub tree.



14.4 Traversing a Binary Tree:

The process of visiting each node of the tree exactly once is called *tree traversal*. The traversal of a binary tree involves three basic activities such as:

1. Visiting the Root
2. Traverse the left sub-tree
3. Traverse the right sub-tree

The three basic activities listed above are done in different order. There are three types of traversal orders followed. They are:

1. Pre-Order Traversal
2. In-Order Traversal
3. Post-Order Traversal

The above tree traversal methods can be implemented in two ways. They are:

1. Recursive Tree Traversal
2. Non-Recursive Tree Traversal

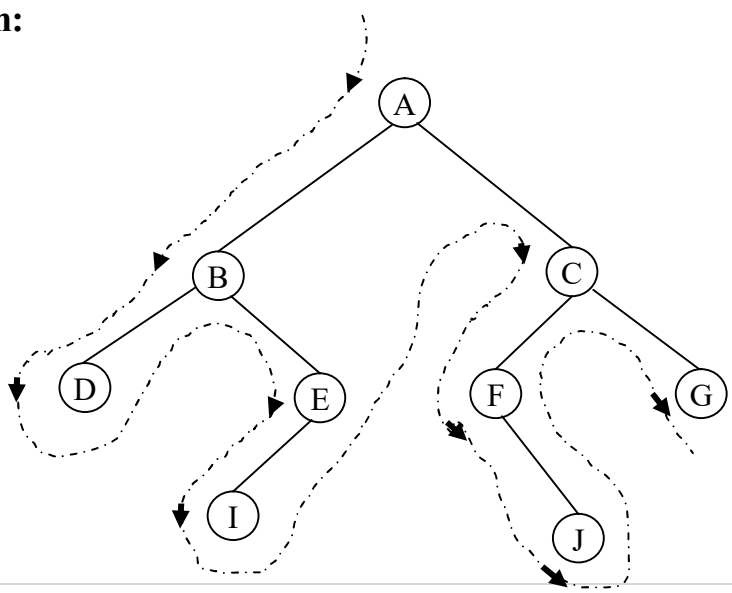
14.5 RECURSIVE TREE TRAVERSAL METHODS:

14.5.1 Preorder Traversal: (NLR) [N- Root Node, L – Left Sub tree, R – Right Sub tree]

The tree will be traversed in the following order:

1. Visit the root.
2. Traverse the left sub-tree in preorder.
3. Traverse the right sub-tree in preorder.

Diagram:



14.5.1.1 Algorithm for Preorder Traversal:

PreOrder(Node)
[Assume a binary tree in memory, node holds the address of the root node of the tree]

Step 1: If Node ≠ NULL, Then
Step 1.1: Process (Node)
Step 1.2: PreOrder(***left***[Node])
Step 1.3: PreOrder(***right***[Node])
[End of If]
Step 2: Exit

14.5.2 InOrder Traversal: (LNR)

The tree will be traversed in the following order:

1. Traverse the left sub-tree in inorder.
2. Visit the root.
3. Traverse the right sub-tree in inorder.

Diagram:

14.5.2.1 Algorithm for InOrder Traversal:

InOrder(Node)
[Assume a binary tree in memory, Node holds the address of the root node of the tree **]**

Step 1: If Node ≠ NULL, Then
Step 1.1: InOrder(***left***[Node])
Step 1.2: Process (Node)
Step 1.3: InOrder(***right***[Node])
[End of If]
Step 2: Exit

14.5.3 PostOrder Traversal: (LRN)

The tree will be traversed in the following order:

1. Traverse the left sub-tree in postorder.
2. Traverse the right sub-tree in postorder.
3. Visit the root.

Diagram:

14.5.3.1 Algorithm for PostOrder Traversal:

PostOrder(Node)
[Assume a binary tree in memory, Node holds the address of the root node of the tree **]**

Step 1: If Node ≠ NULL, Then
Step 1.1: PostOrder(***left***[Node])
Step 1.2: PostOrder(***right***[Node])
Step 1.3: Process (Node)
[End of If]
Step 2: Exit

14.6 NON-RECURSIVE TREE TRAVERSAL METHODS:

Non-recursive procedures for tree traversal will be implemented by using stack.

14.6.1 Algorithm for PreOrder Tree Traversal

PreOrder (ROOT)
[Assume a binary tree in the memory. ROOT is the pointer that holds the address of root node of the tree **]**

```
Step 1: Let PTR: =ROOT
Step 2: Repeat While PTR ≠ NULL
            Step 2.1: PROCESS INFO [PTR]
            Step 2.2: If RIGHT [PTR] ≠ NULL, Then
                          PUSH (RIGHT [PTR])
                      [End of If]
            Step 2.3: If LEFT [PTR] ≠ NULL, Then
                          PTR: =LEFT [PTR]
            Step 2.4: Else
                          PTR: =POP ( )
                      [End of if]
        [End of While]
Step 3: Exit
```

14.6.2 Algorithm for InOrder Tree Traversal

InOrder (ROOT)
[Assume a binary tree in the memory. ROOT is the pointer that holds the address of root node of the tree **]**

```
Step 1: Set PTR: =ROOT
Step 2: Repeat While PTR ≠ NULL
            Step.2.1: PUSH (PTR)
            Step 2.2: PTR: =LEFT [PTR]
        [End of While]
Step 3: Set PTR: =POP ( )
Step 4: Repeat While PTR ≠ NULL
            Step 4.1: PROCESS   INFO [PTR]
            Step 4.2: If RIGHT [PTR] ≠ NULL, Then
                          PTR: =RIGHT [PTR]
```

```
                    GOTO Step 2
                [End of If]
        Step 4.3: PTR: =POP ( )
    [End of While]
Step 5: Exit
```

14.6.3 Algorithm for PostOrder Tree Traversal

PostOrder (ROOT)
[Assume a binary tree in the memory. ROOT is the pointer that holds the address of root node of the tree **]**

```
Step 1: Let PTR: =ROOT
Step 2: Repeat While PTR ≠ NULL
        Step 2.1: PUSH (PTR)
        Step 2.2: If RIGHT [PTR] ≠ NULL, Then
                    PUSH (- RIGHT [PTR])
                [End of If]
        Step 2.3: PTR: =LEFT [PTR]
    [End of While]
Step 3: PTR: =POP ( )
Step 4: Repeat While PTR > 0
        Step 4.1: PROCESS INFO [PTR]
        Step 4.2: PTR: =POP ( )
    [End of While]
Step 5: If PTR< 0, Then
        Step 5.1: Set PTR: = - PTR
        Step 5.2: GOTO Step 2
    [End of If]
Step 6: Exit
```

14.7 CONSTRUCTION OF A BINARY TREE:

A binary tree can be constructed by using a combination of tree traversal methods such as:

1. PreOrder and InOrder traversal
2. PostOrder and InOrder traversal

14.7.1 Construction of Binary Tree using PREORDER and INORDER traversal:

Procedure:

1. 1st node of the PREORDER traversal is the Root Node of the Tree.
2. Find the position of the Root Node in the INORDER traversal.

    a) Nodes Preceding the ROOT in the INORDER construct the left sub tree.
    b) Nodes succeeding the ROOT in the INORDER construct the right sub tree.

3. Find out the next ROOTs for each sub trees using PREORDER
4. Continue the above process until leaf nodes are found.

Example:

Construct a binary tree from the given preorder and inorder tree traversal.

PreOrder : A B D E F C G H J L K
InOrder : D B F E A G C L J H K
Left Subtree Right Subtree

Diagram:

1. The first node the preorder traversal is the root node the tree. In our example, **'A'** is the root node.
2. Find the position of root node **'A'** in inorder traversal. The nodes preceding the root node in the inorder traversal are the nodes of the ***left subtree*** of the root node *i.e.* **DBFE** and the nodes succeed to the root node are the nodes in the ***right subtree*** of the root node **GCLJHK.**

3. Now take two sets of inorder and preorder traversals of the left and right subtrees of the root. The first set is the nodes appear precede to the root node in inorder traversal *i.e.***DBFE** and the combination of those nodes only appear in preorder traversal just after the root node *i.e.* **BDEF.** The second set is the nodes appear after the root node in inorder traversal *i.e.* **GCLJHK** and the nodes in he preorder traversal except the root node and the nodes considered in the first set *i.e.* **CGHJLK.**
4. Considering these two sets of preorder and inorder traversals for the left and right subtrees we can repeat steps 2 and 3 till the entire tree is constructed.

14.7.2 Construction of Binary Tree using POSTORDER and INORDER traversal:

Procedure:

1. Here last Node in the POSTORDER traversal is considered as ROOT Node of the Tree.
2. Find the position of ROOT node in the INORDER traversal
   a. Nodes Preceding the ROOT in the INORDER construct the left sub tree.
   b. Nodes succeeding the ROOT in the INORDER construct the right sub tree.

3. Find out the next ROOTs for each sub trees using POSTORDER
4. Continue the above process until leaf nodes are found.

Example:
Construct a binary tree from the given postorder and inorder traversal.

PostOrder : D F E B G L J K H C A
InOrder : D B F E A G C L J H K
Left Subtree Right Subtree

Diagram:

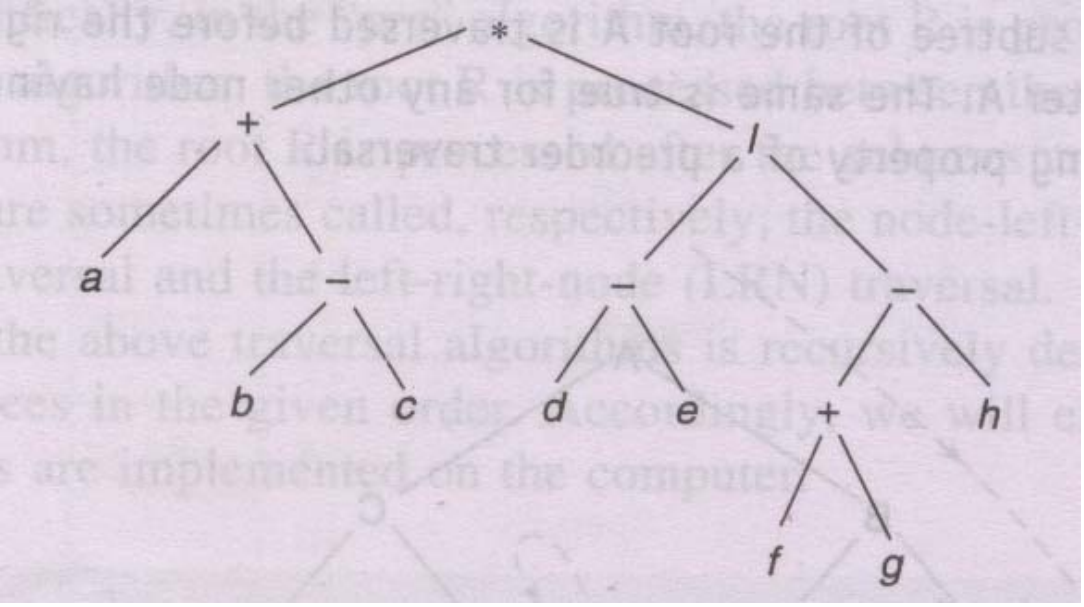
1. The last node the postorder traversal is the root node the tree. In our example, **'A'** is the root node.
2. Find the position of root node **'A'** in inorder traversal. The nodes preceding the root node in the inorder traversal are the nodes of the ***left subtree*** of the root node *i.e.* **DBFE** and the nodes succeed to the root node are the nodes in the ***right subtree*** of the root node **GCLJHK.**
3. Now take two sets of inorder and postorder traversals of the left and right subtrees of the root. The first set is the nodes appear preceding to the root node in inorder traversal *i.e.***DBFE** and the combination of those nodes only appear in postorder traversal just before the root node *i.e.* **DFEB.** The second set is the nodes appear after the root node in inorder traversal *i.e.* **GCLJHK** and the nodes in the postorder traversal except the root node and the nodes considered in the first set *i.e.* **GLJKHC.**
4. Considering these two sets of preorder and inorder traversals for the left and right subtrees we can repeat steps 2 and 3 till the entire tree is constructed.

EXPRESSION TREE

15.1 EXPRESSION TREE

It is a binary tree which stores an arithmetic expression. The leaf nodes of the expression tree are always ***operands*** and all the other internal nodes are the ***operators*** including the root.

For Example: (a + b – c) * ((d – e) / (f + g – h))



Divide and conquer technique used to convert an express into a binary tree.

e.g A + (B + C * D + E) + F/G

<u>Procedence</u>

1. Brackets – inner most - Highest
2. ^ (or) ^ - Exponent – next highest
3. * and / = Middle
4. + and - = lowest level.

15.2 Procedure for Constructing an Expression Tree

1. Given an Infix Expression

2. Find out the Pre fix (or) Post Fix Notation
3. 1st term of the Prefix is the ROOT
4. Identify ROOT in the infix and identify left and right subtree
5. find the next Roots again using infix.
6. Continue the process till leafs exist.

Note : Here we will be finding all the leaf Nodes are the operands and all the internal Nodes are the Operators.

e.g.(1) In Fix : (A+B *(D-E/F))
Prefix : <u>+A*B-D/EF</u>

15.3 Construction of an expression tree from a postfix expression

Procedure:

Step 1: Scan each symbol from the Postfix expression.
Step 2: If an operand is found, then create a one node tree and push a pointer to it onto the stack.
Step 3: If an operator is found, then POP two pointers belonging to two trees say T_1 and T_2 from stack and construct a tree where ROOT is the operator, T_2 is left sub tree and T_1 is right sub tree respectively.
Step 4: PUSH the pointer to the above resultant sub tree onto STACK.
Step 5: Exit

Example:
Construct and expression tree for the following Postfix Expression.

A B C * + D E * F + G / -

<u>DIAGRAM:</u>

BINARY SEARCH TREE

16.1 Binary Search Tree

A binary tree T is termed as binary search tree if each node *n* of T satisfies the following property.
1. The value at *n* is greater than the values of all nodes in its left sub tree and
2. The value at *n* is less than the values of all nodes in its right sub tree.

A binary search tree is useful for binary search.

16.2 Construction Procedure of Binary Search Tree

While constructing a BST, initially we store the first node as ROOT Node, and then we follow insertion of a Node based on the rules of BST. So for each insertion we need to search for appropriate node either left or right based on the value of the node to be inserted.

e.g. Construct a BST with the following input values
70, 20, 11,45, 95, 76, 29, 59, 99

DIAGRAM

Algorithm for insertion into BST

INSERT_BST (ROOT, LEFT, RIGHT, INFO, ITEM, AVAIL)

Step 1: [Allocates the Memory for FRESH node and initialize INFO part of node with ITEM]
FRESH: =AVAIL
INFO [FRESH]:= ITEM
LEFT [FRESH]:=NULL
RIGHT [FRESH]: =NULL

Step 2: If ROOT=NULL, Then

```
                ROOT: =FRESH
Step 3:         Else
                PTR: =ROOT
                Repeat While PTR ≠ NULL
                        PREV: =PTR
                        If INFO [PTR]>INFO [FRESH], Then
                                PTR: =LEFT [PTR]
                        Else
                                PTR: =RIGHT [PTR]
                        [End of If]
                [End of While]
Step 4: If PTR=NULL, Then
        Step 4.1: If INFO [PREV]>INFO [FRESH], Then
                        LEFT [PREV]:=FRESH
        Step 4.2: Else
                        RIGHT [PREV]:=FRESH
                  [End of If]
        [End of If]
Step 5:         EXIT
```

Example:

AVL TREE

17.1 Height Balanced Tree / AVL Tree

In a binary search tree, the effectiveness of searching process always depends upon how data is organized to make up a specific tree.

DIAGRAM:

Search for an element may require more comparisons in **Fig(b)** as compared to **Fig(a)** in the above tree representation. When a binary search tree is not balanced, the operations like searching, insertion, deletion takes considerable time as compared to a binary search tree which is balanced.

Balance factor of a node in binary search tree is the difference between height of left and right sub tree of it.

$$\boxed{BF = H_L - H_R}$$

Where BF = Balance Factor

H_L = Height of left sub-tree

H_R = Height of right sub-tree

So, a binary search tree is height balanced if all its nodes have a balance factor of -1, 0 or 1. So for every node in a height balanced tree **BF Є [-1, 0, 1].**

Assume a tree is balanced initially and the tree becomes imbalanced due to insertion or deletion of a node. Now in order to make the tree balanced we need to follow certain rotation methods. The rotation methods are proposed by Russian Mathematicians G.M. **A**delson, **V**elskii and E.M.**L**andis and accordingly the rotations are named as AVL rotations. So height balanced tree is also called AVL tree.

There are four types of AVL rotations. They are:

1. LL Rotation
2. RR Rotation
3. LR Rotation
4. RL Rotation.

Pivot Node:

Pivot node of a binary search tree is the node where the tree gets imbalanced i.e. the node for which the balance factor BF > 1 is treated as pivot node.

LL Rotation:

When the tree gets imbalanced due to insertion of a node at left of the left sub tree of pivot node, we follow LL rotation.

DIAGRAM:

RR Rotation

When tree gets imbalanced due to insertion of a node at right of the right subtree of pivot node, we follow RR rotation.

DIAGRAM:

LR Rotation

When tree gets imbalanced due to insertion of a node at right of the left subtree of pivot node, we follow LR rotation.

DIAGRAM:

RL Rotation

When tree gets imbalanced due to insertion of a node at left of the right subtree of pivot node, we follow RL rotation.

DIAGRAM:

Example on Building an AVL Tree

17.2 Symbol Table Construction

Every compiler maintains its own symbol table, which contains symbols and their associated value/information. A symbol table can be maintained using linked list(i.e. linked dictionary), but searching of a symbol will be time consuming process.

A symbol table can also be maintained using Binary Search Tree Structure, where searching can be efficient. The required operations on the symbol table are:

1. Insertion of new entry
2. Deletion of an existing entry
3. Searching for an existing entry

Here as the entries of symbol table are uniformly distributed according to the lexicographic order (or) dictionary order, to find Binary Search can be followed. At it maintains lexicographic order, during insertion or deletion of entry, only few pointers need to be changed.

17.3 Applications of Tree

1. To implement file system of several popular OS
2. It can be used to represent and evaluate arithmetic expressions
3. Supports Binary search with time complexity O(log n)
4. Used for symbol table construction.